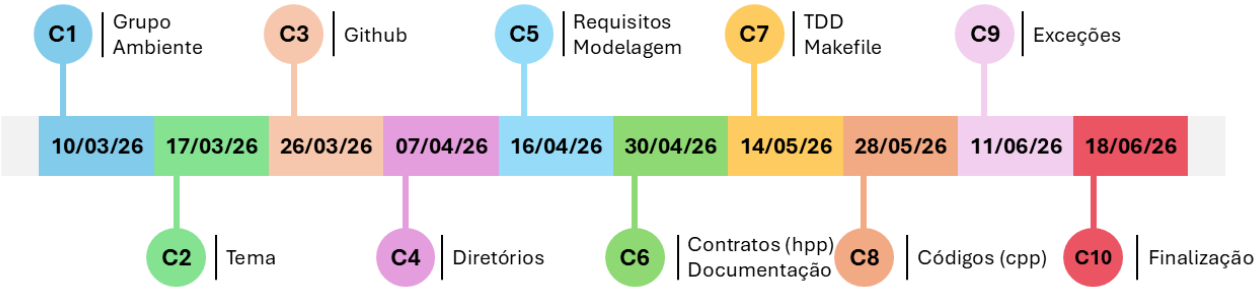
Projeto Final – Detalhamento Checkpoints

O projeto será desenvolvido de forma **incremental**, possibilitando a evolução contínua da solução ao longo do semestre. O acompanhamento do trabalho ocorrerá por meio de **10 checkpoints**, distribuídos ao longo da disciplina, cada um associado a entregáveis específicos que permitem monitorar o progresso e realizar ajustes quando necessário. Os detalhes de cada checkpoint encontram-se descritos a seguir e no cronograma oficial da disciplina. O não cumprimento de um checkpoint acarretará **penalização de 5% na nota final**, cumulativa a cada etapa não atendida.



C1. Formação do Grupo e Configuração do ambiente de desenvolvimento

Cada grupo deve preparar e validar o ambiente de desenvolvimento, garantindo que todos os integrantes tenham instalados e configurados os compiladores, bibliotecas, IDEs e demais ferramentas necessárias. O objetivo é assegurar que não haverá obstáculos técnicos para o início do projeto. Além disso, o grupo deve preencher as informações dos integrantes na Planilha do Moodle.

C2. Definição do tema

Os grupos deverão escolher o tema do projeto e informar na Planilha do Moodle.

C3. Acesso ao GitHub e Commit de teste

Cada integrante do grupo deverá acessar o repositório no Github e realizar um commit de teste. Primeiro faça a correta configuração local de seu usuário, considerando:

```
git config user.name "O SEU NOME COMO ESTA NO MOODLE"
git config user.email "O SEU EMAIL COMO ESTA NO MOODLE"
```

Em seguida, criar um arquivo de texto na raiz do projeto com nome **[sua matrícula].txt**, e adicione o seguinte conteúdo no arquivo.

Estou ciente que a avaliação da participação final será baseada única e exclusivamente na quantidade de commits realizada por mim. Entendo que não serão contabilizados commits feitos em desacordo com os dados de usuário cadastrados acima. A correta divisão de tarefas entre os membros é responsabilidade minha e de meu grupo.

C4. Estrutura de diretórios

O grupo deve organizar a estrutura inicial de diretórios do repositório, incluindo pelo menos as pastas `src/`, `include/`, `build/`, `tests/` e `design/`. O objetivo é adotar uma padronização que facilite a modularização do projeto.

C5. Requisitos e Modelagem

Nesta etapa, o grupo deve levantar os requisitos por meio de **User Stories** e elaborar a modelagem inicial utilizando **Cartões CRC**. Esses artefatos devem ser armazenados no diretório `design/`.

As User Stories[1] são uma forma simples de apresentar os requisitos funcionais desejados para um determinado sistema. São artefatos de desenvolvimento utilizados principalmente em processos baseados em metodologias ágeis. As descrições são intencionalmente genéricas, dando liberdade ao grupo para decidir detalhes da implementação.

O grupo deve identificar funcionalidades interessantes para o sistema e propor pelo menos **SEIS** User Stories. Para cada uma, deverá ser feita uma descrição sucinta, similar ao exemplo abaixo.

- **Sistema de Gestão Acadêmica**
- **Descrição:**
  - Como professor quero gerar um relatório para verificar o desempenho dos alunos.
- **Critérios de aceitação:**
  - Mostrar a pontuação da avaliação atual de um aluno.
  - Mostrar a pontuação de avaliação passada de um aluno.
  - Mostrar a média e desvio padrão da turma para a avaliação.

Tente apresentar User Stories para **diferentes tipos de usuário do sistema** (ou considerando papeis específicos em certos contextos). Cada User Story deve apresentar entre **3 e 5 critérios de aceitação**.

Em seguida, faça pelo menos **SEIS** cartões CRC[2] para as classes que você julga como sendo as mais importantes no sistema. Cada cartão deve apresentar entre **5 e 10 responsabilidades** (considerando conhecimento e realização) e mencionar pelo menos **2 colaborações**. Você pode usar esse site para lhe auxiliar a fazer o cartão: https://h3ct0r.github.io/crcmaker/

C6. Definição dos contratos (`.hpp`) e documentação inicial (Doxygen+README)

Os arquivos de cabeçalho devem ser definidos com as assinaturas das funções e classes que comporão o sistema, baseados na modelagem feita na etapa anterior. Paralelamente, deve-se iniciar a documentação usando o Doxygen[3], descrevendo a estrutura proposta e funcionalidade dos métodos.

---

[1] https://en.wikipedia.org/wiki/User_story
[2] https://en.wikipedia.org/wiki/Class-responsibility-collaboration_card
[3] http://www.doxygen.org/

Além disso, o arquivo README[4] do repositório deve ser atualizado com informações básicas, como o nome dos integrantes do grupo, a descrição inicial do problema a ser tratado, seus objetivos principais e a motivação para a escolha (1 a 2 parágrafos), e demais dados relevantes para contextualização do projeto.

C7. Testes de unidade (TDD) e Makefile

O grupo deve começar a implementar testes de unidade para o sistema seguindo a abordagem *Test-Driven Development* (TDD[5]). Nesse momento, é esperado que os testes ainda não passem, já que os comportamentos ainda não foram implementados. O objetivo é estruturar o processo de verificação futura do código. O grupo deverá utilizar o framework doctest[6] para implementar os testes de unidade. Deve haver pelo menos uma classe de testes para cada uma das principais classes do sistema (por exemplo, as classes de entidades). Além disso, ao final, deve-se apresentar uma cobertura total do código de pelo menos **60%** (a ser verificado utilizando a ferramenta gcovr[7]).

Também deve ser criado um *Makefile* para automatizar a compilação, garantindo reprodutibilidade.

C8. Implementação inicial dos comportamentos (`.cpp`)

As primeiras versões das funcionalidades devem ser implementadas nos arquivos `.cpp`, em conformidade com os contratos definidos nos cabeçalhos. O objetivo é validar a integração entre os módulos e iniciar a construção efetiva da solução.

C9. Programação defensiva e Tratamento de exceções

O código deve ser refinado para incluir práticas de programação defensiva, validando entradas, prevenindo erros comuns e implementando tratamento adequado de exceções. Essa etapa busca aumentar a robustez e a confiabilidade do sistema.

A interface de interação com a aplicação poderá ser feita via terminal de comando. Possíveis arquivos necessários durante a etapa de inicialização do sistema deverão ser fornecidos pelo grupo.

C10. Finalização e Apresentação

O grupo deve consolidar a versão final do projeto, realizando ajustes, revisões de código e testes finais. Além da entrega do código-fonte e da documentação atualizada, será feita uma apresentação para a turma, demonstrando o funcionamento da solução.

---

[4] https://docs.github.com/pt/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes
[5] https://en.wikipedia.org/wiki/Test-driven_development
[6] https://github.com/onqtam/doctest
[7] https://gcovr.com/en/stable/